

A store operating system for the paint trade.

StoreERP runs the full counter-to-compliance loop for a paint & hardware retailer: product catalogue, perpetual inventory, purchase and sales billing, AI-assisted invoice capture, and Indian GST accounting — all under one multi-tenant roof with a system-admin control tower on top.

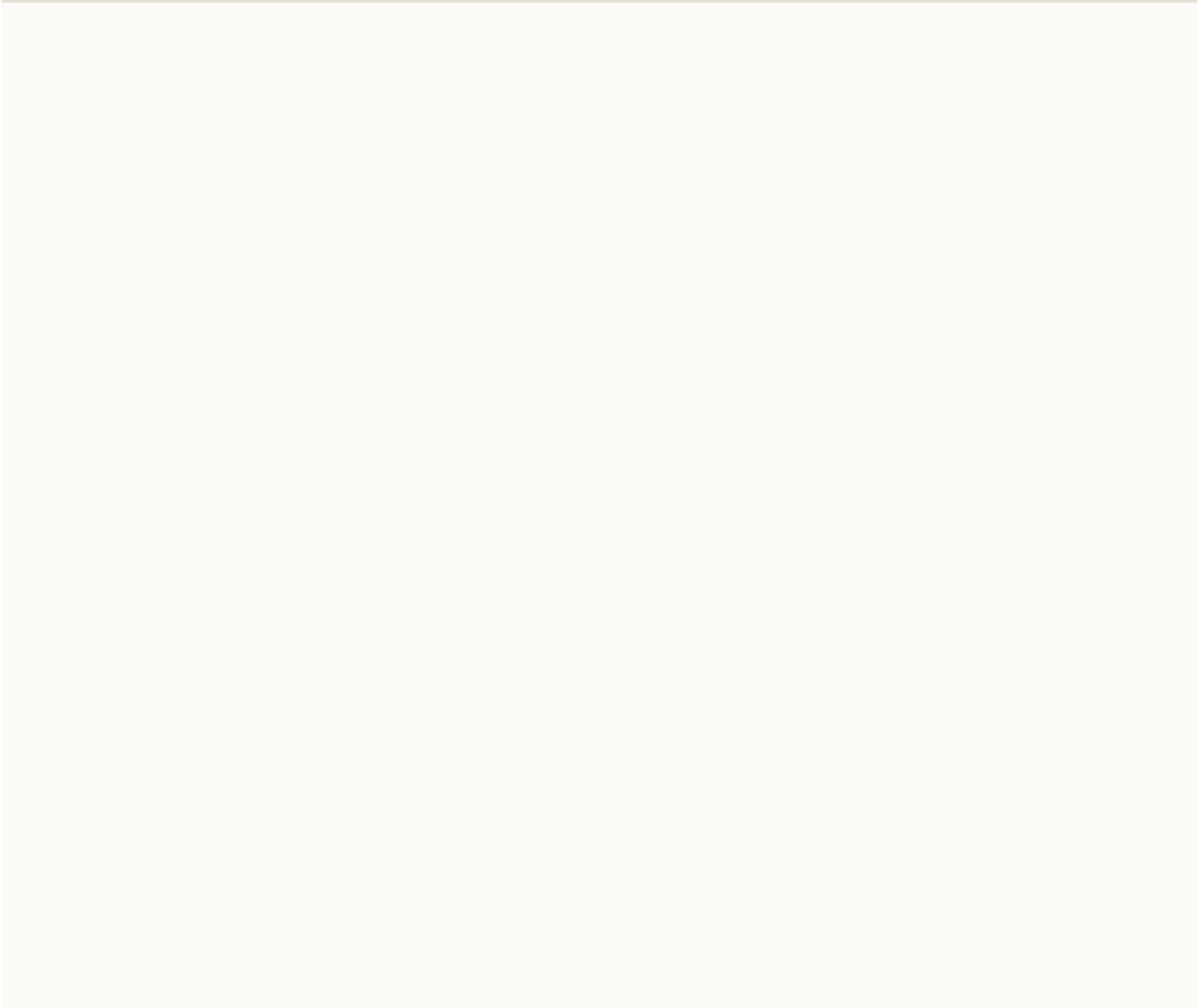
Stack Ruby on Rails 7.1 · PostgreSQL

Jobs GoodJob

Tables 37

Tenancy org-scoped

AI Gemini / Groq vision



01 System overview

The platform is organised as one shared product universe with per-store books. A central catalogue of paint products, brands, shades and units of measure is maintained once; each **organisation** (store) layers its own pricing, stock, suppliers, customers, invoices and tax ledgers on top.

T Multi-tenancy

Every transactional row carries an `organisation_id`. Stores never see each other's stock, billing or ledgers. Shared masters (`products`, `brands`, `shade_catalogues`) are linked to a store through `organisation_products`.

R Roles

Four-tier access: `staff` `admin` `owner` `super_admin`. Staff operate the counter; owners manage a store; super-admin governs the whole platform & catalogue.

₹ Compliance-native

GST is not bolted on — intra/inter-state tax splits, HSN codes, input-credit ledgers and GSTR-1 / 3B summaries are first-class parts of the invoice lifecycle.

🏠 Paint domain

Purpose-built for paint retail: shade catalogues, tinting-machine canister tracking, pack codes and brand price-lists are modelled directly, not shoe-horned into generic SKUs.

THE SEVEN FUNCTIONAL MODULES

MODULE	ROUTE	NAMESPACE	WHAT IT OWNS
Setup	/setup		Products, categories, brands, UOMs, shade catalogues, price-lists, imports
Inventory	/inventory		Stock levels, opening stock, stock ledger, tinting machine
Purchasing	/purchasing		Suppliers, purchase invoices, AI digitise, payments, payables
Sales	/sales		Customers, referrers, sales invoices, receipts, receivables
Accounting	/accounting/gst		GSTR-1, GSTR-3B, HSN summary, ITC, period close
Reports	/reports		Sales, purchases, stock, cash-flow — all exportable
Admin	/admin		System processes, product review queue, performance

02 Data model & ERD

Thirty-seven tables, grouped into seven concerns. The entity map below shows the transactional spine — how a store, its masters, and its documents connect. Job-queue and import-log tables are omitted for clarity.

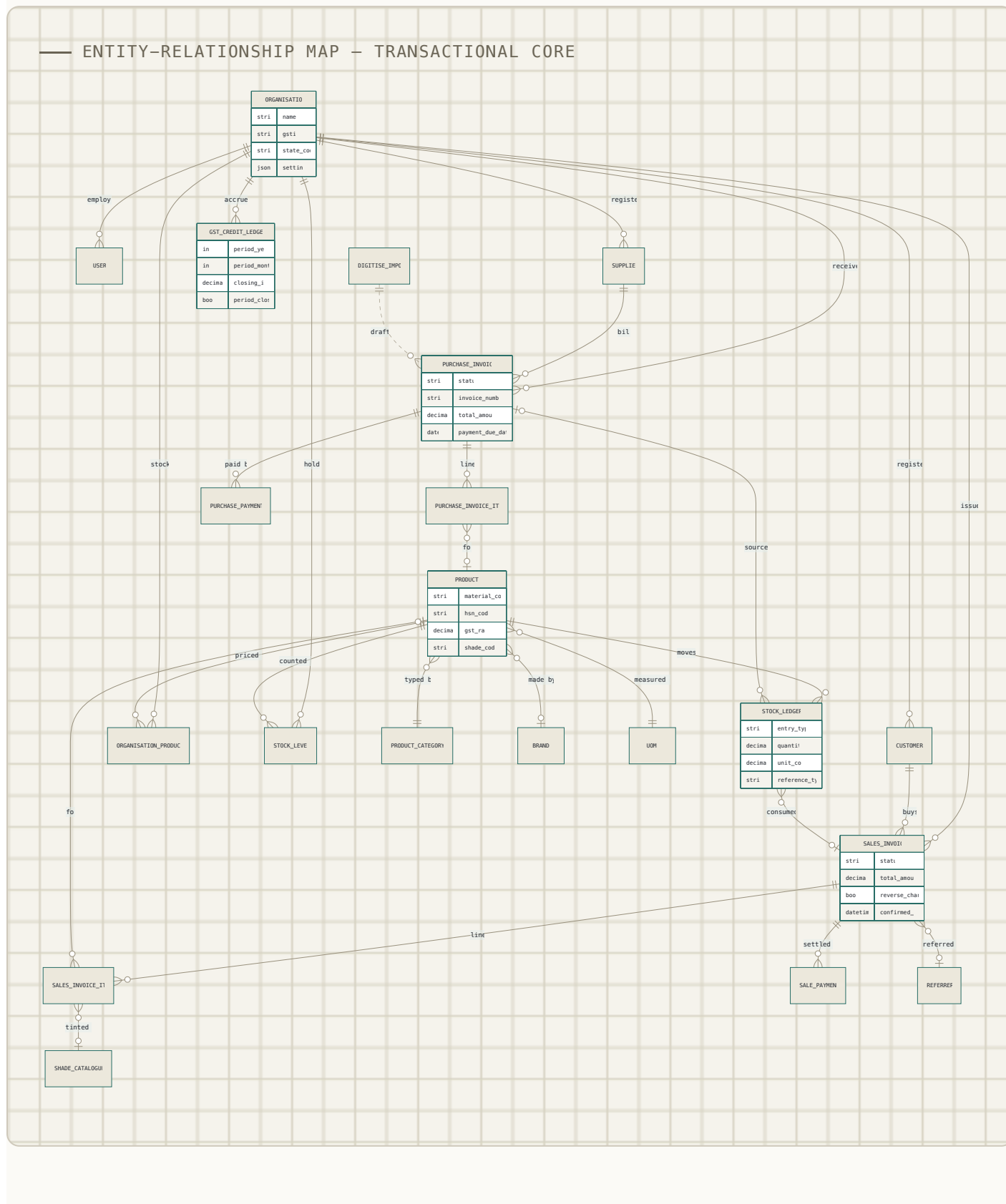


TABLE FAMILIES

CONCERN	TABLES
Identity & tenancy	organisations, users, activity_logs
Catalogue masters	products, organisation_products, product_categories, brands, uoms, shade_catalogues
Inventory	stock_levels, stock_ledgers, tinting_machine_canisters, tinting_machine_logs
Purchasing	suppliers, purchase_invoices, purchase_invoice_items, purchase_payments, digitise_imports
Sales	customers, referrers, sales_invoices, sales_invoice_items, sale_payments
GST & accounting	gst_credit_ledger_entries
System & ops	external_api_logs, good_jobs & queue tables, *_imports, product_merge_logs

03 Inventory & stock ledger

Inventory is **perpetual and event-sourced**. Two tables cooperate: `stock_levels` is the fast running balance (quantity + moving-average cost) per product, and `stock_ledgers` is the immutable append-only history of every movement that produced that balance.

`stock_levels`

The live balance

One row per product per store. Holds `quantity` and `avg_cost`. Read by the counter, reports and low-stock views. Recomputed, never guessed.

`stock_ledgers`

The audit trail

Every movement is a signed row with an `entry_type` and a polymorphic link back to the document that caused it (`reference_type` / `reference_id`).

LEDGER ENTRY TYPES

`purchase` + stock in `sale` - stock out `opening` initial count `adjustment`
± corrections & void reversals

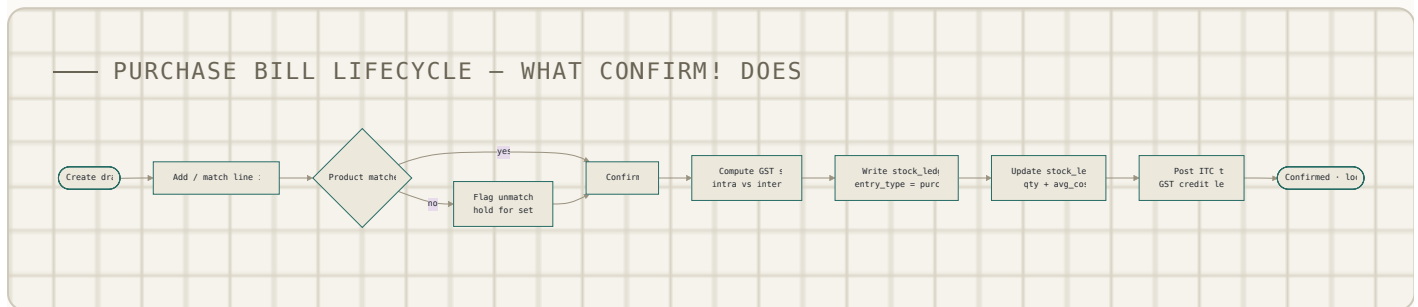
Because the balance is derived from the ledger, every quantity on the shelf is **traceable to a source document** — a confirmed purchase, a sale, an opening count, or a manual adjustment. Voiding a sale doesn't delete history; it writes a compensating `adjustment` that returns the goods. The `/inventory/stock_levels` screen adds search, filtering, card/list views, quick-adjust and CSV export; opening stock is captured through its own guided entry at `/inventory/opening_stock/new`.

Paint-specific: the tinting-machine module tracks colourant **canisters** and dispensing **logs** per brand, so base + tinter consumption is visible alongside finished-goods stock.

04 Purchase bills

A purchase invoice records goods received from a supplier. It lives as a **draft** — fully editable, no side-effects — until it is **confirmed**, which is the single moment stock and tax are posted.

draft → review & edit lines **confirm!** → **confirmed**



TAX SPLIT AT CONFIRMATION

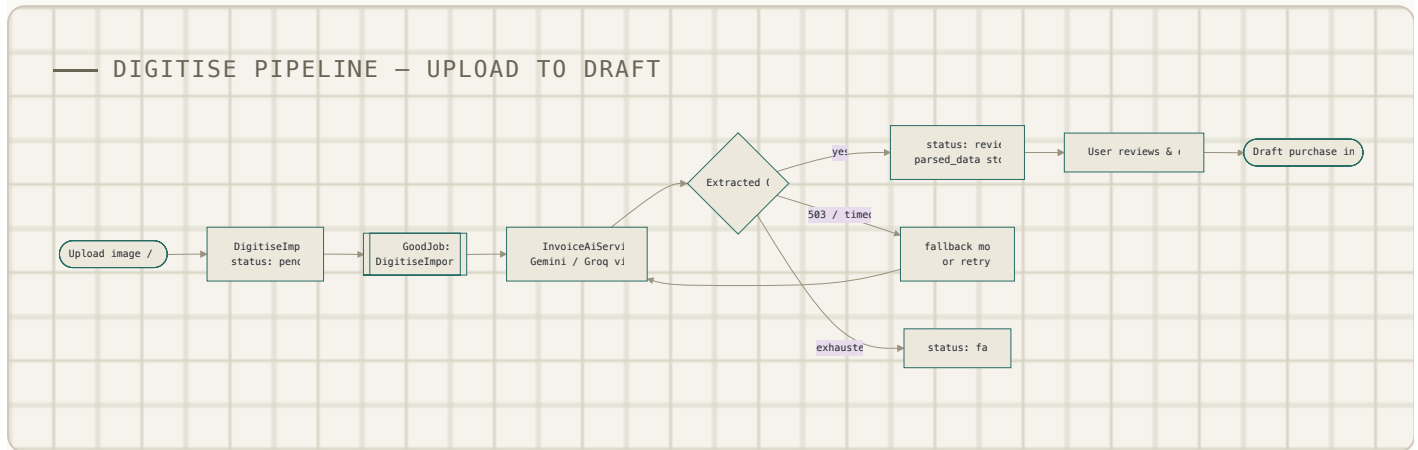
The model compares the **supplier's state** against the store's state. Same state → the GST rate is split into **CGST** + **SGST** (**intra_state**). Different state → the whole rate becomes **IGST** (**inter_state**). Per-line **cgst_amount** , **sgst_amount** , **igst_amount** and **taxable_amount** are all computed and stored so the invoice is self-contained for audit.

PAYMENTS & PAYABLES

- **purchase_payments** record money paid to suppliers (default mode **bank_transfer**), each with a reference number and date.
- **Payment status** is derived — **unpaid** / **partial** / **paid** — from payments against invoice total.
- **Accounts payable** at **/purchasing/accounts_payable** ages outstanding balances against **payment_due_date** .

05 AI invoice capture

Rather than key in a supplier bill by hand, a user photographs or uploads it. A vision model reads the image and returns structured line items, which land in a **review** queue before becoming a draft purchase invoice — a human always confirms.



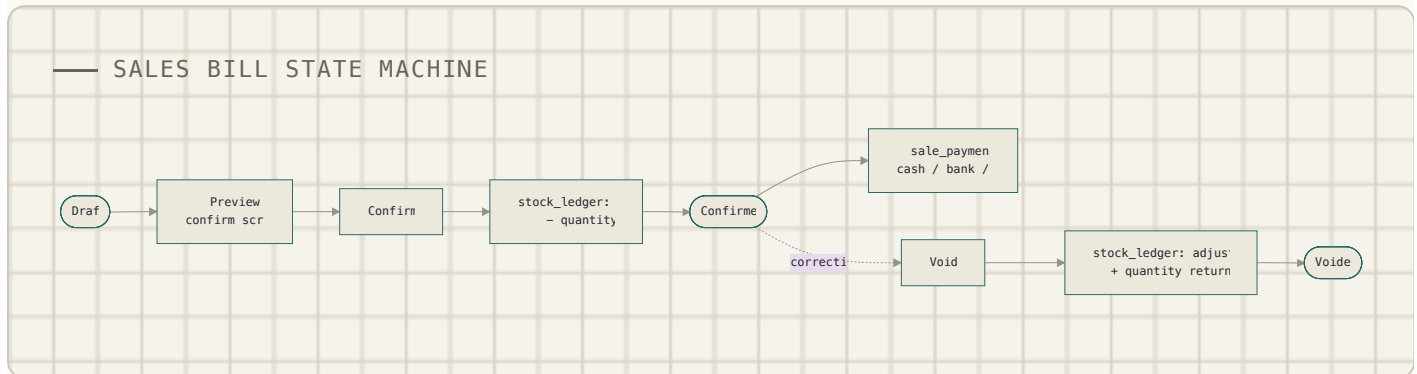
PROVIDER STRATEGY

The parser is provider-agnostic behind `InvoiceAiService`. Google **Gemini** handles full-resolution bills on the free tier and walks a model fallback chain (`gemini-flash-lite-latest` → `gemini-3.5-flash` → `gemini-flash-latest`) so a single overloaded model doesn't fail the scan. Groq's Qwen vision model is available as an alternative. Every call is written to `external_api_logs` with duration and status, so the admin tower sees scan health in real time.

Design principle: AI output is a **draft proposal**, never a posted document. Extraction feeds a review step; stock and tax move only when a person confirms the resulting purchase bill.

06 Sales bills

Sales invoices mirror purchases in reverse: `draft` → `preview` → `confirmed`, with a `voided` terminal state for corrections. Confirmation depletes stock; voiding returns it.



🕒 Tinted line items

A sales line can reference a **shade catalogue** entry plus a base product and a tinter product — capturing exactly what was dispensed at the counter, not just a generic SKU.

% Discounts & RC

Per-line and invoice-level (`overall_discount_amount`) discounts, plus a `reverse_charge` flag, feed the same intra/inter-state GST computation as purchases.

➡ Referrers

Optional **referrer** attribution per invoice supports painter/contractor referral tracking — a real driver in paint retail.

₹ Receivables

`sale_payments` (default `cash`) settle invoices; `/sales/accounts_receivable` ages what customers still owe against due dates.

07 GST & taxation

Because every confirmed line stores its own tax breakdown, the accounting module is largely a **summarisation layer**. It rolls confirmed purchases (input credit) and confirmed sales (output tax) into the statutory returns and runs the monthly input-tax-credit set-off.

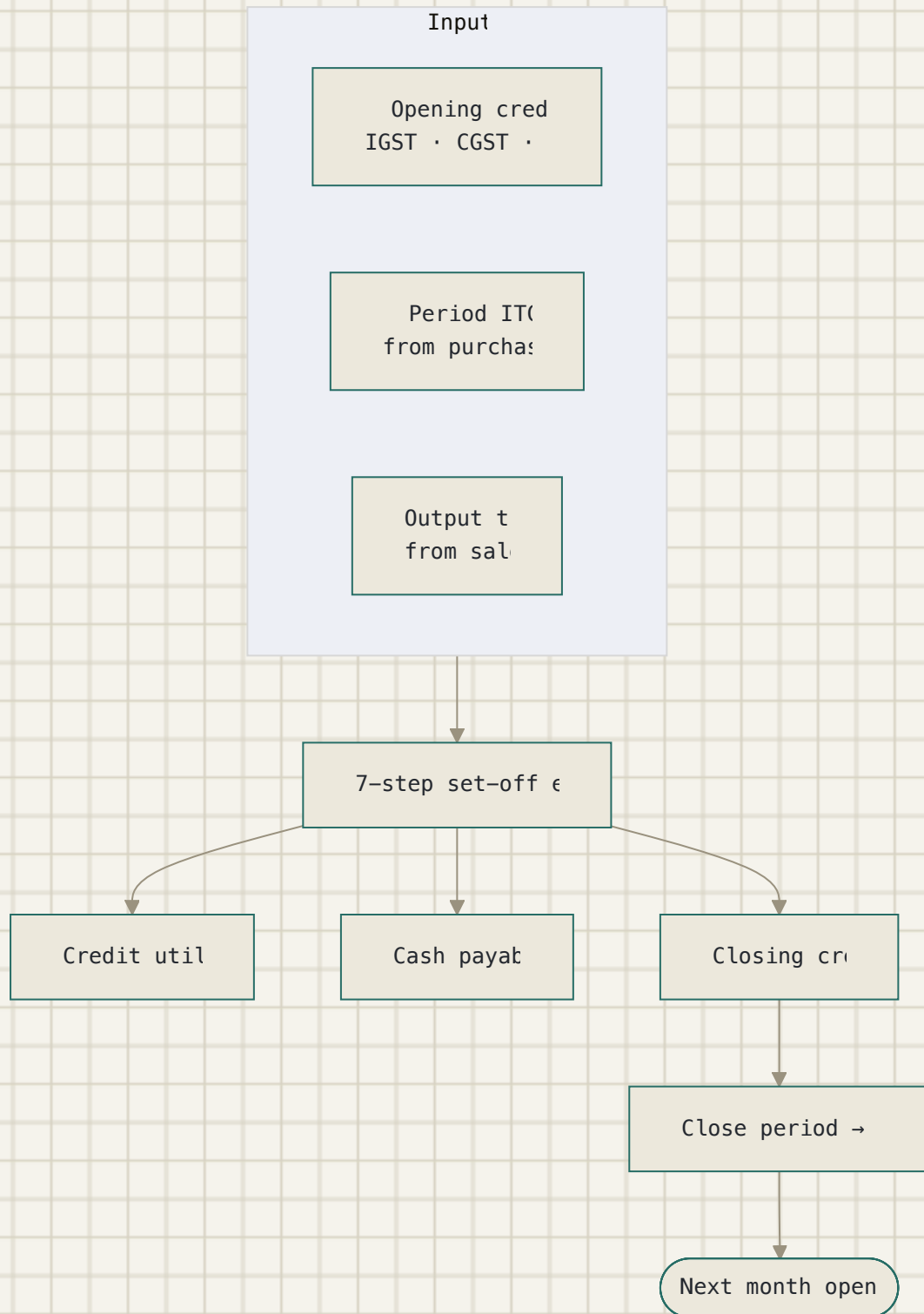
STATUTORY OUTPUTS – /ACCOUNTING/GST

REPORT	SOURCE	PURPOSE
GSTR-1	Confirmed sales	Outward supplies + HSN summary of what was sold
GSTR-3B	Sales vs purchases	Output tax, eligible ITC, and net tax payable after set-off
ITC report	Purchase items	Input credit available, grouped by GST rate
HSN summary	All lines	Quantity & value rolled up by HSN code

INPUT-TAX-CREDIT LEDGER & PERIOD CLOSE

The `gst_credit_ledger_entries` table keeps a per-month (`period_year` / `period_month`) running position across all three tax heads — opening credit, credit earned in the period, output tax, and the closing balance carried forward. GSTR-3B runs a **seven-step ITC set-off** (IGST first, then CGST/SGST in the statutory order) to compute cash payable versus credit utilised. Closing a period locks it (`period_closed` , `closed_at` , `closed_by`) and rolls the closing credit forward as next month's opening.

— MONTHLY ITC SET-OFF & CARRY-FORWARD



08 Reporting

Four operational reports sit under `/reports`, each with a matching CSV export for the accountant. They read the same confirmed documents the ledgers do, so numbers reconcile by construction.

`/reports/sales`

Sales report

Revenue by period, customer and product, with tax and discount breakdowns.

`/reports/purchases`

Purchases report

Spend by supplier and product; feeds cost and ITC visibility.

`/reports/stock_reports`

Stock report

On-hand quantity and valuation at moving-average cost, per product.

`/reports/cash_flows`

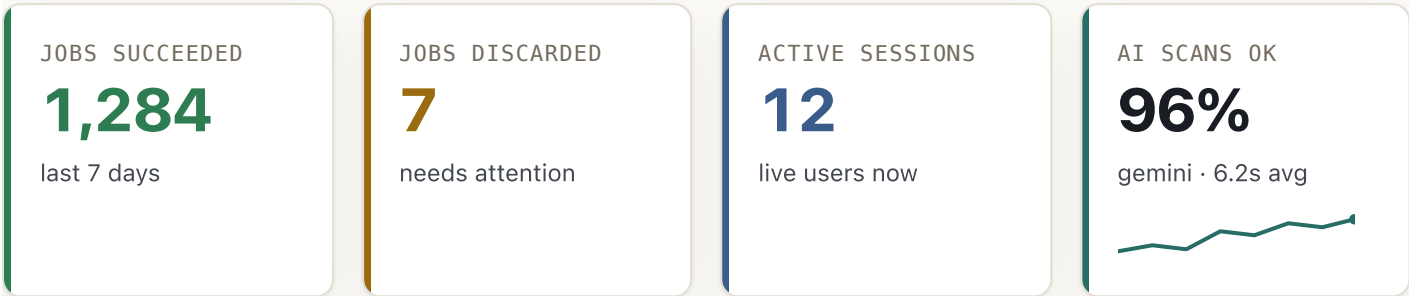
Cash flow

Money in (receipts) versus money out (supplier payments) over time.

The store **dashboard** (`/dashboard`) gives the owner a daily read: month-to-date sales and purchase trend charts plus payables coming due — the numbers a store runs on each morning.

09 Admin control tower

Under Setup, the super-admin gets a platform-wide operations console at `/admin/system_processes` — a single screen to watch background work, external-AI health, live sessions and product-data quality across every store. Illustrative snapshot:



WHAT THE TOWER TRACKS

PANEL	BACKED BY	ANSWERS
Background jobs	<code>good_jobs</code>	What's queued, running, succeeded or discarded — filterable by class and date
External API log	<code>external_api_logs</code>	Every AI/vendor call with service, operation, HTTP status & duration
Active users	<code>users.session_token</code>	Who is logged in right now; force-logout available
Job-class stats	<code>good_jobs</code> agg.	Throughput & failure rate per job type
Product filters	per-org catalogue	Catalogue coverage and filter build status by store
Product review	<code>/admin/product_reviews</code>	Approve, reject, merge or apply AI suggestions to catalogue entries

Every AI invoice scan, price-list import and enrichment job flows through these same logs, so the fix we ship on any provider is **observable end-to-end** from one console — success rate, latency and failure reason, per service.



Working today

Catalogue, multi-tenant stores, perpetual inventory, purchase & sales billing with GST posting, payments, AR/AP, GSTR-1/3B, AI invoice capture (Gemini), reports & admin tower.



Hardening

Feature-test coverage across purchasing / sales / inventory; provider resilience for AI capture; mobile-responsive views & PWA shell.



Next

Deeper cash-book accounting, e-invoice / e-way-bill hooks, and richer per-store analytics on the owner dashboard.